DAA Practicals

Design and Analysis of Algorithms: complete theory, code explanation and viva question bank

Source: github.com/imanasig/DAA (6 Python programs covering 5 practicals). Every program was run and its output verified while preparing this document, so the dry runs and expected outputs below are real.

How to use this document

- Part 0 gives the big picture: paradigms, comparison and a complexity cheat sheet. Read it once before the viva.
- Each practical (1 to 6) follows the same order: **Problem, Theory, Approach, Code walkthrough, Dry run, Features and edge cases, Complexity, Why this design, Viva Q&A**.
- The Q&A sections are written as the examiner would chain them: question, answer, then the natural follow-up ("why?", "what if?").
- Last section lists weak points of the code that an examiner might probe, with the honest answer ready.

The six programs at a glance

#	File	Problem	Paradigm	Time complexity
1	Fibonacci_Hanoi.py	Fibonacci (recursive vs iterative) and Tower of Hanoi	Recursion	Fib rec: $O(2^n)$; Fib iter: $O(n)$; Hanoi: $O(2^n)$
2	Huffman_Coding.py	Build Huffman tree, encode, decode	Greedy + min-heap	O(k log k), k = distinct characters
3	Fractional_Knapsack_Greedy.py	Max profit when items can be split	Greedy	O(n log n)
4	01Knapsack_DP.py	Max profit, items all-or-nothing	Dynamic Programming	O(nW) time and space
5	01Knapsack_BranchNbound.py	Same 0/1 knapsack via bounding	Branch and Bound	Exponential worst case
6	NQueens.py	N queens with the first queen fixed by user	Backtracking	O(n!) worst case

How to run (Linux / any OS with Python 3)

```
python3 --version          # check Python 3 is installed
cd path/to/DAA
python3 Fibonacci_Hanoi.py   # same pattern for the other five files
```

Only the standard library is used (heapq in Huffman, `time` in Fibonacci). No pip install is needed. Each program asks for input in the terminal.

Part 0: Big picture

0.1 The five paradigms used in this repository

Paradigm	One-line idea	Needs	Used in
Recursion	Solve a problem by calling the same function on a smaller input until a base case.	Base case + smaller subproblem	Fibonacci, Tower of Hanoi
Greedy	At every step pick the locally best choice and never undo it.	Greedy-choice property + optimal substructure	Huffman, Fractional Knapsack
Dynamic Programming	Store answers of overlapping subproblems in a table and reuse them.	Optimal substructure + overlapping subproblems	0/1 Knapsack (DP)
Branch and Bound	Explore the solution tree but cut off a branch when its best possible value cannot beat the best answer found.	A bounding function (upper bound for maximise)	0/1 Knapsack (B&B)
Backtracking	Build the solution step by step; undo the last step when a dead end is reached.	A feasibility (constraint) check	N-Queens

0.2 The same data solved three ways (the knapsack triangle)

Items (value, weight): (60,10), (100,20), (120,30). Capacity = 50. This single example is the best way to explain why three different knapsack algorithms exist.

Method	Result	Items taken	Comment
Fractional knapsack (greedy by ratio)	240.00	Item 1, item 2, and 2/3 of item 3	Splitting allowed, so greedy is optimal.
Greedy by ratio applied to 0/1 (wrong idea)	160	Item 1, item 2 (item 3 does not fit whole)	Leaves 20 units of capacity empty. Not optimal.
0/1 Knapsack, DP	220	Item 2, item 3	True optimum when no splitting is allowed.
0/1 Knapsack, Branch and Bound	220	Item 2, item 3	Same optimum, found by pruned search.

Remember: Fractional answer (240) is always ≥ 0/1 answer (220). That is exactly why the fractional solution is used as the **upper bound** inside Branch and Bound.

0.3 Complexity cheat sheet

Algorithm	Time	Space	Note
Fibonacci recursive	$O(2^n)$ (tight: $\Theta(\varphi^n)$, $\varphi \approx 1.618$)	O(n) stack depth	Recomputes the same subproblems
Fibonacci iterative	O(n)	O(1)	Keeps only last two values
Tower of Hanoi	$O(2^n)$ exactly $2^n - 1$ moves	O(n) stack	Output itself is exponential
Huffman coding	O(k log k) build, O(k) codes, O(b) decode	O(k)	k = distinct chars, b = bits
Fractional knapsack	O(n log n)	O(n)	Sorting dominates
0/1 Knapsack DP	O(nW)	O(nW)	Pseudo-polynomial; 1-D array gives O(W) space
0/1 Knapsack B&B	Exponential worst case (about $O(n \cdot 2^n)$ here)	O(n)	Often much faster in practice
N-Queens backtracking	O(n!) worst case (≈ O(n·n!) with O(n) safety check)	$O(n^2)$ board + O(n) stack	Pruning makes it far faster than brute force

0.4 General viva questions (any practical)

Q1. What is an algorithm? What properties must it have?

A finite sequence of well-defined steps that solves a problem. Properties: input, output, definiteness (each step clear), finiteness (it terminates), effectiveness (each step is doable).

Q2. What is time complexity and why do we use Big-O?

Time complexity measures how the number of basic operations grows with input size n. Big-O gives an upper bound (worst-case growth) and ignores constants and machine speed, so algorithms can be compared fairly.

Q3. Difference between O, Ω and Θ?

O is an upper bound, Ω a lower bound, Θ a tight bound (both). Example: recursive Fibonacci is $O(2^n)$ as a loose upper bound but $\Theta(\varphi^n)$ exactly.

Q4. What is optimal substructure?

An optimal solution to the problem contains optimal solutions to its subproblems. Required by both Greedy and DP.

Q5. What are overlapping subproblems?

The same subproblem is solved many times. DP exploits this by storing results. Divide and conquer (merge sort) has non-overlapping subproblems.

Q6. Greedy vs DP, how do you decide?

If a locally best choice is provably safe (greedy-choice property), use greedy: it is simpler and faster. If choices interact and you must compare several options, use DP. Fractional knapsack is greedy; 0/1 knapsack needs DP.

Q7. Backtracking vs Branch and Bound?

Backtracking prunes a branch because it is *infeasible* (violates constraints), used for decision/enumeration problems like N-Queens. Branch and Bound prunes a branch because it is *not better than the best known* answer, used for optimisation problems like knapsack.

Q8. What does pseudo-polynomial mean?

Running time is polynomial in the numeric value of an input (such as W) but exponential in its bit-length. DP knapsack O(nW) is pseudo-polynomial.

Q9. Is 0/1 knapsack NP-hard?

Yes, the optimisation version is NP-hard (decision version is NP-complete). It is only weakly NP-hard because the pseudo-polynomial DP exists.

Q10. Why Python, and why menus and input validation?

Python gives short readable code for algorithm logic. Validation (negative values, empty input, wrong type) makes the programs robust and shows edge-case handling, which examiners like.

Practical 1: Fibonacci (recursive vs iterative) and Tower of Hanoi

File: Fibonacci_Hanoi.py | **Paradigm:** Recursion

1.1 Problem statement

- Generate the Fibonacci sequence using a recursive and a non-recursive method; compare execution time and complexity.
- Solve Tower of Hanoi for a given number of disks using recursion; count the moves.

1.2 Theory

- **Recursion:** a function that calls itself on a smaller input. Needs a *base case* (stops) and a *recursive case* (reduces the problem). Each call sits on the call stack, so depth = space used.
- **Fibonacci:** F(0)=0, F(1)=1, F(n)=F(n−1)+F(n−2). Sequence: 0 1 1 2 3 5 8 13 ...
- **Why naive recursion is slow:** fib(5) calls fib(4) and fib(3); fib(4) calls fib(3) again. The same values are recomputed again and again (overlapping subproblems). Recurrence: T(n)=T(n−1)+T(n−2)+1, which grows like $\varphi^n \approx 1.618^n$.
- **Tower of Hanoi:** 3 pegs (source, auxiliary, destination), n disks stacked small on large. Rules: move one disk at a time; never place a larger disk on a smaller one. Recurrence T(n)=2T(n−1)+1 ⇒ T(n)=2^n−1 moves.

1.3 Approach

1. **Recursive Fibonacci:** if n ≤ 1 return n, else return fib(n−1)+fib(n−2).
2. **Iterative Fibonacci:** keep two variables (n1=0, n2=1); in a loop print n1 and update (n1, n2) = (n2, n1+n2).
3. **Hanoi:** (a) move n−1 disks source→auxiliary using destination as helper; (b) move disk n source→destination; (c) move n−1 disks auxiliary→destination using source as helper.

1.4 Code walkthrough

```
def fib_recursive(n):
    if n <= 1:            # base cases: fib(0)=0, fib(1)=1
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)

def fib_non_recursive(n):
    n1, n2 = 0, 1
    for i in range(n):
        print(n1, end=" ")
        n1, n2 = n2, n1 + n2     # simultaneous update (tuple assignment)
    print()

def tower_of_hanoi(n, source, auxiliary, destination, move_count):
    if n == 0: return
    tower_of_hanoi(n-1, source, destination, auxiliary, move_count)   # step (a)
    move_count[0] += 1
    print(f"Move disk {n} from {source} to {destination}")            # step (b)
    tower_of_hanoi(n-1, auxiliary, source, destination, move_count)   # step (c)
```

- **main():** reads number of terms (allowed 0 to 35) and disks (allowed 0 to 15); prints both Fibonacci series; times each with `time.perf_counter()`; prints complexity text; then runs Hanoi and prints total moves against the expected 2^n−1.
- The recursive Fibonacci section calls `fib_recursive(i)` for every i in 0..n−1 to print the series, so its total work is the sum of all those calls, still exponential.
- **move_count = [0]**: a one-element list is used as a mutable counter because Python integers are immutable and cannot be updated through a function argument.
- Time is multiplied by 1,000,000 to display microseconds because perf_counter returns seconds.
- **try/except ValueError** catches non-integer input.

1.5 Dry run

Fibonacci with n = 7 → **0 1 1 2 3 5 8** (both methods). Hanoi with 3 disks (real program output), 7 moves = 2^3-1:

Move	1	2	3	4	5	6	7
Disk: from → to	1: A→C	2: A→B	1: C→B	3: A→C	1: B→A	2: B→C	1: A→C

1.6 Features and edge cases handled

- Input range check protects against freezing: recursive Fibonacci at n=35 already needs tens of millions of calls; Hanoi with 15 disks prints 32,767 lines.
- n = 0 gives an empty sequence and 0 moves (base case n == 0).
- Non-integer input handled by ValueError.
- Prints measured time *and* theoretical complexity side by side so the difference is visible.

1.7 Complexity

Method	Time	Auxiliary space
Recursive Fibonacci	$O(2^n)$ upper bound (exactly $\Theta(\varphi^n)$)	O(n): recursion depth
Iterative Fibonacci	O(n)	O(1)
Tower of Hanoi	$O(2^n)$; exactly 2^n-1 moves	O(n): recursion depth

1.8 Why was it done this way?

- Placing both Fibonacci versions together demonstrates that the *same* output can have hugely different cost, motivating memoisation and DP (Practical 4).
- Hanoi shows a problem whose structure is naturally recursive and whose optimum is provably exponential.
- Improvements to mention: memoisation or bottom-up DP → O(n); fast doubling or matrix exponentiation → O(log n).

1.9 Viva Q&A

Q1. What is recursion? What are its parts?

A function calling itself on a smaller instance. Parts: base case (stops recursion) and recursive case. Without a base case you get infinite recursion and a stack overflow (Python's default limit is about 1000 frames).

Q2. Why is recursive Fibonacci slow?

It recomputes identical subproblems exponentially many times. Recurrence $T(n)=T(n-1)+T(n-2)+1$. **Follow-up: how slow?** $\Theta(\varphi^n)$ with $\varphi=1.618$; $O(2^n)$ is the simple upper bound because $T(n-2) \le T(n-1)$.

Q3. Why is its space O(n) when it makes exponentially many calls?

Space counts how many frames exist *at the same time*, not total calls. The deepest chain is fib(n)→fib(n−1)→...→fib(1), depth n.

Q4. Why is iterative Fibonacci O(1) space?

Only two variables are kept and updated. (If you stored the whole series in a list it would be O(n).)

Q5. How can you make recursive Fibonacci efficient?

Memoisation (store computed fib values in a dictionary) or bottom-up DP gives O(n). Fast doubling or matrix power gives O(log n).

Q6. Why use perf_counter instead of time.time?

perf_counter is a high-resolution monotonic clock meant for measuring short intervals; time.time can have coarse resolution and can jump if the system clock changes.

Q7. What is the minimum number of moves in Tower of Hanoi, and how do you prove it?

2^n-1. From $T(n)=2T(n-1)+1$ with $T(0)=0$, or by induction: $T(n)=2(2^{n-1}-1)+1=2^n-1$. Moving the largest disk requires the other n−1 disks to be on the auxiliary peg first, so no shorter way exists.

Q8. Can Hanoi be solved in less than exponential time?

No. The number of required moves is itself exponential, so even printing the answer takes exponential time.

Q9. Why O(n) space for Hanoi?

Only the recursion stack (depth n). The peg names and counter use constant space.

Q10. What is the role of move_count = [0]?

Mutable counter shared by all recursive calls. Alternatives: a global variable, *nonlocal*, or returning the count from the function.

Q11. In Hanoi, why are source, auxiliary and destination swapped in the recursive calls?

Each sub-problem plays the same game with different roles. Step (a) wants n−1 disks to end on the auxiliary peg, so auxiliary becomes the "destination" and destination becomes the helper. Step (c) swaps roles again.

Q12. Which disk moves most often?

Disk 1 (smallest) moves 2^{n-1} times; disk k moves 2^{n-k} times.

Q13. Is recursive Fibonacci divide and conquer?

It looks like it but the subproblems overlap, so it is really a DP problem solved inefficiently. Merge sort is true divide and conquer (independent halves).

Q14. What is tail recursion? Does Python optimise it?

A call that is the last action of the function, which some compilers turn into a loop. Python does not do tail-call optimisation.

Q15. Why limit n to 35 and disks to 15?

Practical limits: recursive Fibonacci becomes very slow beyond about 35-40, and Hanoi prints 2^n-1 lines (about 32 thousand at 15).

Practical 2: Huffman Coding

File: Huffman_Coding.py | **Paradigm:** Greedy with a min-heap (priority queue)

2.1 Problem statement

Given a text, build Huffman codes for its characters, encode the text into a binary string, and decode a binary string back to text.

2.2 Theory

- **Goal:** lossless data compression. Instead of a fixed 8 bits per character (ASCII), frequent characters get short codes and rare ones get long codes.
- **Prefix code:** no code is a prefix of another (e.g. A=0, C=10, B=11). This makes decoding unambiguous without separators. If A=0 and B=01 the string "001" would be ambiguous.
- **Greedy choice:** repeatedly merge the two least frequent nodes. Justification: in some optimal tree the two rarest symbols are siblings at the deepest level, so merging them first is safe.
- **Cost** of a code = Σ (frequency × code length). Huffman minimises this among all prefix codes.
- The tree is a *full binary tree*: k leaves (characters) ⇒ k−1 internal nodes.

2.3 Approach (as implemented)

1. Count frequency of each character in a dictionary.
2. Push (frequency, counter, character) for every character into a min-heap.
3. While the heap has more than one node: pop the two smallest, create a new node (left, right) with frequency f1+f2, push it back.
4. The last node left is the root of the Huffman tree.
5. Traverse the tree: going left appends '0', going right appends '1'; at a leaf store the code.
6. **Encode:** concatenate the code of every character. **Decode:** walk from the root following bits; at a leaf output the character and restart from the root.

2.4 Code walkthrough

```
def build_tree(text):
    freq = {}
    for ch in text: freq[ch] = freq.get(ch, 0) + 1
    heap = []; count = 0
    for ch in freq:
        heapq.heappush(heap, (freq[ch], count, ch)); count += 1   # (frequency, tie-breaker, node)
    while len(heap) > 1:
        f1, _, left  = heapq.heappop(heap)
        f2, _, right = heapq.heappop(heap)
        heapq.heappush(heap, (f1 + f2, count, (left, right))); count += 1
    return heap[0][2]

def generate_codes(root, code, codes):
    if isinstance(root, str):              # leaf = a single character
        codes[root] = code if code else "0"   # single-character text gets code "0"
        return
    generate_codes(root[0], code + "0", codes)
    generate_codes(root[1], code + "1", codes)
```

- **Node representation:** a leaf is just the character (a string); an internal node is a tuple (left, right). `isinstance(root, str)` tells them apart. No Node class is needed.
- **Why the counter in the tuple?** If two entries have equal frequency, Python would next compare the third element. Comparing a string with a tuple raises TypeError. The unique counter always breaks the tie first and also makes the output deterministic.
- **encode()** returns None if a character is not in the code table (reported as "Character not found in tree!").
- **decode()** validates: only '0'/'1' allowed; if the bits end in the middle of a path (current != root) the input is invalid; special branch when the tree is a single leaf (only '0's allowed, output = char × number of bits).

- **Menu loop:** 1 Build tree, 2 Encode, 3 Decode, 4 Exit. Encoding and decoding refuse to run until a tree has been built.

2.5 Dry run (verified)

Text **AAABBC**: frequencies A=3, B=2, C=1.

1. Pop C(1) and B(2) → new node (C,B) with frequency 3.
2. Heap now has A(3) and node(3). Counter breaks the tie, A is popped first (left), node second (right) → root with frequency 6.

```
        (6)
       /   \
   A(3)    (3)          Codes:  A = 0,  C = 10,  B = 11
          /   \
       C(1)   B(2)      Encoded AAABBC = 000 1111 10 = 000111110  (9 bits)
                        Fixed 8-bit ASCII would need 6 x 8 = 48 bits
```

Other verified runs: **AAAAA** → A gets code 0, encoded text 00000. **HELLO WORLD** → 32 bits instead of 88; decoding gives back HELLO WORLD exactly. Different tie-breaking can give different codes, but the total encoded length is always the same (optimal).

2.6 Complexity

Step	Time	Why
Build tree	O(k log k)	k−1 merges, each does 2 pops + 1 push on a heap of size ≤ k (O(log k) each)
Generate codes	O(k)	One visit per tree node (2k−1 nodes)
Encode	O(n + output length)	One dictionary lookup per input character
Decode	O(b)	One step per input bit
Space	O(k)	Heap, tree and code table

2.7 Viva Q&A

Q1. What is Huffman coding?

A greedy algorithm that builds an optimal prefix code for lossless compression, giving shorter codes to more frequent characters.

Q2. Why is it a greedy algorithm?

At each step it commits to merging the two smallest frequencies and never reconsiders. This is safe by the greedy-choice property and gives a globally optimal tree.

Q3. What is a prefix code and why do we need it?

No codeword is a prefix of another. It lets the decoder know exactly where one character ends without delimiters.

Q4. Why a min-heap?

We need to repeatedly extract the minimum. A heap does extract-min and insert in O(log k); a sorted list would need O(k) per insert.

Q5. Why is there a counter inside the heap tuple?

Tie-breaker for equal frequencies; prevents Python from comparing a str with a tuple (TypeError) and keeps results deterministic.

Q6. Are Huffman codes unique?

No. Ties and left/right choice can produce different trees. But all of them give the same minimum total length.

Q7. What is the time complexity?

O(k log k) to build the tree for k distinct characters.

Q8. What happens with a one-character text?

The tree is a single leaf; its path is empty, so the code is set to "0" (otherwise the code length would be 0 and nothing could be encoded).

Q9. Is Huffman lossy or lossless?

Lossless: decode(encode(text)) = text.

Q10. How much did you compress in your example?

AAABBC: 9 bits vs 48 bits with 8-bit ASCII. (Even vs a 2-bit fixed code for 3 symbols = 12 bits, Huffman is shorter.)

Q11. What are the drawbacks?

The decoder needs the tree or frequency table; two passes over data (count then encode); static model; arithmetic coding or adaptive Huffman can do better.

Q12. Where is it used in real life?

DEFLATE (ZIP, gzip, PNG), JPEG and MP3 entropy-coding stages.

Q13. Huffman vs Shannon-Fano?

Huffman builds bottom-up and is always optimal; Shannon-Fano splits top-down and is not always optimal.

Q14. What is the longest possible Huffman code?

k−1 bits, for very skewed (Fibonacci-like) frequencies, giving a chain-shaped tree.

Q15. What if all characters have the same frequency?

The tree becomes balanced; code lengths are about log2(k), like a fixed-length code.

Practical 3: Fractional Knapsack (Greedy)

File: Fractional_Knapsack_Greedy.py | **Paradigm:** Greedy

3.1 Problem statement

Given n items with profit (value) v_i and weight w_i, and a knapsack of capacity W, choose amounts x_i between 0 and 1 of each item to maximise $\Sigma\, v_i x_i$ subject to $\Sigma\, w_i x_i \le W$. Fractions of an item are allowed.

3.2 Theory

- **Greedy criterion:** value-to-weight ratio (profit per unit weight). Take the item with the highest ratio first.
- **Why it is optimal (exchange argument):** suppose an optimal solution uses some weight on a lower-ratio item while a higher-ratio item is not fully used. Swap that weight to the better item: profit does not decrease. So there is an optimal solution that fills items in ratio order.
- Because splitting is allowed, capacity is never wasted: the knapsack is completely filled unless all items fit.
- Other criteria fail: highest value first or lightest first can both be beaten (see Q2).

3.3 Approach

1. Compute ratio = value / weight for each item.
2. Sort items in descending order of ratio.
3. Scan the sorted list: if the whole item fits, take it and reduce remaining capacity.
4. If it does not fit, take the fraction remaining_capacity / weight, add value × fraction, and stop.

3.4 Code walkthrough

```
class Item:
    def __init__(self, item_id, value, weight):
        self.id, self.value, self.weight = item_id, value, weight
        self.ratio = value / weight

def fractional_knapsack(items, capacity):
    items.sort(key=lambda x: x.ratio, reverse=True)      # greedy choice
    total_value, remaining_capacity = 0.0, capacity
    for item in items:
        if remaining_capacity <= 0: break
        if item.weight <= remaining_capacity:             # whole item
            total_value += item.value
            remaining_capacity -= item.weight
        else:                                             # fraction
            fraction = remaining_capacity / item.weight
            total_value += item.value * fraction
            remaining_capacity = 0
    return total_value
```

- The **Item class** bundles id, value, weight and ratio so the original item number is not lost after sorting.
- Prints the sorted table and, for each item, the percentage taken (100% or a fraction such as 66.67%).
- **Validation in main():** number of items must be > 0; an item with weight ≤ 0 is skipped (it would give division by zero); capacity must be > 0; values and weights are floats so decimals work.

3.5 Dry run (verified)

Item	Value	Weight	Ratio	Taken	Capacity left	Profit added
1	60	10	6.00	100%	40	60
2	100	20	5.00	100%	20	100
3	120	30	4.00	66.67% (20 of 30)	0	80

Total = 240.00. Second test (all items fit): values 10,20,30, weights 2,3,4, capacity 20 → all taken, total 60.

3.6 Complexity

- Sorting: O(n log n) (dominates). Scan: O(n). **Total time O(n log n)**. Space O(n) for the item list.
- Can be done in O(n) using a weighted-median selection instead of full sorting (advanced).

3.7 Viva Q&A

Q1. Why is greedy optimal for the fractional knapsack?

Greedy-choice property via exchange argument: swapping weight from a lower-ratio item to a higher-ratio item never reduces profit. Since fractions are allowed, we can always fill remaining space exactly.

Q2. Why ratio and not highest value or smallest weight?

Counter-example for highest value: capacity 20, items A(value 20, weight 20), B(18, 10), C(18, 10). Highest value picks A = 20, but B + C = 36. Ratio is the quantity that measures usefulness per unit of the scarce resource (capacity).

Q3. Why does greedy NOT work for 0/1 knapsack?

Our own data: ratio-greedy takes items 1 and 2 (profit 160) and wastes 20 units because item 3 will not fit whole; the optimum is items 2 and 3 = 220. Without splitting, leftover capacity cannot be used.

Q4. Time complexity and why?

O(n log n) because of sorting; the selection loop is O(n).

Q5. Can you do better than O(n log n)?

Yes, O(n) with a linear-time weighted median (quickselect idea), but sorting is simpler and standard.

Q6. What if all items fit?

The loop takes every item whole; total value = sum of all values (60 in the second test case).

Q7. What if two items have the same ratio?

Either order gives the same total profit, though the selected set can differ.

Q8. Why does the program stop after taking a fraction?

Once a fraction fills the remaining capacity, capacity becomes 0, so nothing else can be added.

Q9. Is the sort stable? Does it matter?

Python's sort (Timsort) is stable, so equal-ratio items keep input order; it does not affect optimality.

Q10. Why use float for value and weight?

Fractions and decimal weights are allowed in this problem; integer-only would break the fractional amounts.

Q11. Real-life uses?

Loading divisible goods (grain, liquids), allocating CPU or bandwidth proportionally, portfolio-style resource allocation where amounts are divisible.

Q12. Relation to Branch and Bound?

The fractional solution is an upper bound on the 0/1 solution, so B&B uses it as its bounding function (Practical 5).

Practical 4: 0/1 Knapsack using Dynamic Programming

File: 01Knapsack_DP.py | **Paradigm:** Dynamic Programming (bottom-up tabulation)

4.1 Problem statement

Given n items with values and weights and capacity W, find the maximum total value such that each item is either taken completely or not at all (0 or 1), and also report which items were chosen.

4.2 Theory

- **Why DP:** the problem has *optimal substructure* (best for first i items and capacity w depends on best for first i−1 items) and *overlapping subproblems* (the same (i, w) state is reached through many different choices). Brute force would try all 2^n subsets.
- **State:** dp[i][w] = maximum value using only the first i items with knapsack capacity w.
- **Recurrence:** for item i with weight w_i and value v_i: if $w_i > w$ then dp[i][w] = dp[i−1][w] (cannot take it); otherwise dp[i][w] = max(dp[i−1][w] (skip), v_i + dp[i−1][w−w_i] (take)).
- **Base cases:** dp[0][w] = 0 (no items) and dp[i][0] = 0 (no capacity). The whole table starts as zeros.
- **Answer:** dp[n][W].

4.3 Approach

1. Create an (n+1) × (W+1) table filled with 0.
2. Fill row by row, column by column using the recurrence.
3. Print the table (for understanding and the viva).
4. Backtrack from dp[n][W] to recover the selected items.

4.4 Code walkthrough

```
dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if weights[i - 1] <= w:
            dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],   # take
                           dp[i - 1][w])                                    # skip
        else:
            dp[i][w] = dp[i - 1][w]

selected = []; w = capacity
for i in range(n, 0, -1):                 # traceback
    if dp[i][w] != dp[i - 1][w]:          # value changed => item i was taken
        selected.append(i)
        w -= weights[i - 1]
selected.reverse()
return dp[n][capacity], selected
```

- **Index shift:** table row i corresponds to item i, stored at list index i−1 (hence weights[i−1]).
- **Traceback logic:** if dp[i][w] equals dp[i−1][w], item i made no difference so it was skipped. If it differs, item i was taken: record it and reduce w by its weight, then continue upward.
- **Validation:** n > 0, value ≥ 0, weight > 0, capacity ≥ 0; everything must be an integer (capacity is used as a table index) otherwise ValueError message. Also prints total weight versus capacity as a sanity check.

4.5 Dry run (verified): weights 1,3,4,5; values 1,4,5,7; capacity 7

i \ w	0	1	2	3	4	5	6	7
0 (none)	0	0	0	0	0	0	0	0
1 (v=1,w=1)	0	1	1	1	1	1	1	1
2 (v=4,w=3)	0	1	1	4	5	5	5	5
3 (v=5,w=4)	0	1	1	4	5	6	6	9
4 (v=7,w=5)	0	1	1	4	5	7	8	9

Answer dp[4][7] = **9**. Traceback: row 4: 9 = 9 → item 4 not taken; row 3: 9 ≠ 5 → item 3 taken, w = 7−4 = 3; row 2: dp[2][3]=4 ≠ dp[1][3]=1 → item 2 taken, w = 0; stop. Items {2, 3}, weight 7, value 9.

Main example (60/10, 100/20, 120/30, W=50): maximum profit **220**, selected items [2, 3], total weight 50.

4.6 Complexity

Measure	Value	Reason
Time	O(nW)	(n+1)(W+1) cells, O(1) work each
Space	O(nW)	Full table is kept so that the selected items can be traced back
Optimised space	O(W)	Single 1-D array, but item recovery is then not straightforward

Remember: O(nW) is **pseudo-polynomial**: W is a number, and its input size is only $\log_2 W$ bits. If W doubles in bit-length, time becomes exponential in the input size.

4.7 Viva Q&A

Q1. What is dynamic programming? How is it different from divide and conquer and greedy?

DP solves overlapping subproblems once and stores the results. Divide and conquer has independent subproblems; greedy commits to one choice without a table.

Q2. Define dp[i][w].

Maximum profit using the first i items with capacity w.

Q3. Write the recurrence.

dp[i][w] = dp[i−1][w] if $w_i > w$, else max(dp[i−1][w], v_i + dp[i−1][w−w_i]).

Q4. Why does the first row and column contain zeros?

With zero items or zero capacity the maximum profit is 0. The loops start from index 1 and rely on this.

Q5. How do you get the selected items?

Backtrack from (n, W): if dp[i][w] ≠ dp[i−1][w] the item was taken, so record it and subtract its weight.

Q6. Time and space complexity?

O(nW) both. Space can be cut to O(W) with a 1-D array.

Q7. Is O(nW) polynomial? Then is knapsack in P?

No. It is pseudo-polynomial: polynomial in the value of W, exponential in the number of bits of W. The 0/1 knapsack problem is NP-hard.

Q8. How does the 1-D optimisation work and why must w run downwards?

Use one array dp[w]; for each item loop w from W down to w_i. Going downwards ensures dp[w−w_i] still holds the previous item's value, so each item is used at most once. Going upwards would allow repeated use (unbounded knapsack).

Q9. What if weights are fractional or W is huge (like 10^9)?

The table becomes infeasible. Use Branch and Bound, or DP indexed by value (dp[v] = minimum weight to reach value v), or scale and round for approximation (FPTAS).

Q10. Why does greedy fail here?

Ratio-greedy picks items 1 and 2 (160) but optimum is items 2 and 3 (220). Choices interact through the capacity, so we must compare options, which DP does.

Q11. Top-down memoisation vs bottom-up tabulation?

Same recurrence. Memoisation (recursion + cache) computes only needed states but uses the call stack; tabulation (this code) is iterative, faster in practice, no recursion limit.

Q12. Can there be more than one optimal solution?

Yes, ties are possible; the traceback returns only one of them.

Q13. What is the brute-force complexity?

$O(2^n)$: every subset is tried.

Q14. Why are floats not allowed in this program?

Capacity indexes the table columns, so values must be integers.

Q15. How is unbounded knapsack different?

Each item can be used any number of times; the recurrence uses $dp[w-w_i]$ in the same row.

Practical 5: 0/1 Knapsack using Branch and Bound

File: 01Knapsack_BranchNbound.py | **Paradigm:** Branch and Bound (depth-first with an upper bound)

5.1 Problem statement

Same 0/1 knapsack problem as Practical 4, solved by a different strategy: search the tree of include/exclude decisions but skip (prune) branches that cannot give a better answer.

5.2 Theory

- **State-space tree:** level i decides item i; left branch = include, right branch = exclude. A full tree has 2^n leaves.
- **Bounding function:** for a node, the best profit any completion of it could possibly reach (an *upper bound* because we maximise). Computed by the fractional-knapsack idea: take the remaining items in ratio order, and fill the leftover space with a fraction of the next item.
- **Pruning rule:** if bound ≤ best profit found so far, no completion can beat the current best, so discard the node.
- **Why the bound is valid:** relaxing the 0/1 constraint to allow fractions can only increase profit, so fractional ≥ any real 0/1 answer. Items are sorted by ratio so this greedy fractional value is the tightest simple bound.
- **Result is still 0/1:** fractions appear only inside the bound calculation, never in the chosen items.

5.3 Approach

1. Sort items by value/weight ratio, descending.
2. Start at the root (item 0, weight 0, profit 0). At every node update best profit if the current profit is better.
3. Compute the upper bound; if it is ≤ best profit, return (prune).
4. Branch 1: include the item (only if it still fits). Branch 2: exclude the item.
5. After the search ends, return the best profit and the best item set.

5.4 Code walkthrough

```
def bound(items, i, weight, profit, capacity):
    total, result = weight, profit
    while i < len(items) and total + items[i][2] <= capacity:   # take whole items
        total += items[i][2]; result += items[i][1]; i += 1
    if i < len(items):                                           # fraction of next item
        result += (capacity - total) * items[i][3]               # items[i][3] = ratio
    return result

def solve(i, weight, profit, selected):
    nonlocal best_profit, best_items
    if profit > best_profit:
        best_profit = profit; best_items = selected.copy()       # copy, list changes later
    if i == len(items): return
    if bound(items, i, weight, profit, capacity) <= best_profit: return   # PRUNE
    item_id, value, wt, ratio = items[i]
    if weight + wt <= capacity:                                  # INCLUDE branch
        selected.append(item_id)
        solve(i + 1, weight + wt, profit + value, selected)
        selected.pop()                                           # undo (backtrack)
    solve(i + 1, weight, profit, selected)                       # EXCLUDE branch
```

- Each item is a tuple (id, value, weight, ratio). The id is kept so the answer refers to original item numbers after sorting; the result is returned as `sorted(best_items)`.
- **selected.copy()** is essential: the same list is later modified by append/pop, so without a copy best_items would silently change.
- **nonlocal** lets the inner function update best_profit and best_items of the enclosing function.
- Best profit is updated at every node, not just at leaves, because any partial selection that fits is a valid solution.

- Include is explored first because high-ratio items are the most promising, which finds a good best_profit early and prunes more later.

5.5 Dry run (verified on the main example)

Sorted order: item 1 (ratio 6), item 2 (5), item 3 (4). Capacity 50. Root bound = 60 + 100 + (20 × 4) = 240.

Step	Node (next item, weight, profit)	Action	Best so far
1	Include 1 → include 2 → (2, 30, 160)	Item 3 does not fit; only exclude branch left	160
2	Include 1 → exclude 2 → (2, 10, 60): bound 60+120 = 180 > 160	Explore; include item 3 → profit 180	180
3	Exclude 1 → (1, 0, 0): bound 100+120 = 220 > 180	Include 2 then include 3 → profit 220, weight 50	220
4	Exclude 1, exclude 2 → (2, 0, 0): bound 120 ≤ 220	**Pruned**	220

Final: profit **220**, items [2, 3], weight 50, identical to the DP answer.

5.6 Complexity

- Sorting O(n log n). Worst-case search is exponential (all 2^n nodes visited if nothing is pruned); each node costs O(n) for bound(), so a safe bound is $O(n \cdot 2^n)$.
- Space O(n) for the recursion stack and the selected list. Best case is far smaller than 2^n when pruning works well.

5.7 Viva Q&A

Q1. What is Branch and Bound?

A systematic search of the solution tree (branching) that uses a bounding function to discard subtrees that cannot contain a better solution than the best found so far.

Q2. How is it different from backtracking?

Backtracking cuts a branch when it breaks a constraint (infeasible). B&B also cuts feasible branches whose upper bound shows they cannot improve the answer. B&B is for optimisation.

Q3. What is your bounding function?

Fractional knapsack value of the remaining items added to the current profit.

Q4. Why is the fractional value a valid upper bound?

Allowing fractions relaxes the problem, and a relaxation can never be worse than the original problem. So no 0/1 completion can exceed it.

Q5. Why sort items by ratio first?

So the greedy fractional computation in bound() is the true fractional optimum, giving the tightest cheap bound; also finds good solutions early.

Q6. Why do you prune when bound ≤ best (not <)?

If the bound only equals the best, the branch can at most tie, never improve; so it is not worth exploring.

Q7. Is this implementation depth-first or best-first?

Depth-first (recursive). Textbook Least-Cost / best-first B&B uses a priority queue ordered by bound and expands the most promising node first. DFS needs only O(n) memory; best-first can prune more but uses more memory.

Q8. Is the answer always optimal?

Yes. Only branches proven unable to beat the current best are discarded, so the optimum is never lost.

Q9. What is the worst-case complexity?

Exponential, $O(2^n)$ nodes ($O(n \cdot 2^n)$ with the bound cost). Example: when all items have equal ratio the bound prunes very little.

Q10. DP or B&B: which is better?

DP is predictable, O(nW), good when W is small. B&B is good when W is huge or weights are not integers, and often prunes heavily, but has no polynomial guarantee.

Q11. Why selected.copy()?

selected is mutated by append/pop during the search. Without a copy, best_items would point to the same changing list.

Q12. Why update the best at every node?

Every node represents a valid partial selection (we only include items that fit), so its profit is achievable.

Q13. Why does the final answer contain no fractions although bound() uses one?

The fraction is only used to estimate; the include branch always takes a whole item.

Q14. What does the state-space tree look like here?

Binary tree of depth n: each level is an item, edges are include/exclude. Pruning removes whole subtrees.

Q15. What if two items have the same ratio?

Any order is fine; both bound validity and optimality are unaffected.

Practical 6: N-Queens with the First Queen Fixed (Backtracking)

File: NQueens.py | **Paradigm:** Backtracking

6.1 Problem statement

Place n queens on an n × n chessboard so that no two queens attack each other (no shared row, column or diagonal). The position of the **first queen is entered by the user** and fixed; place the remaining n−1 queens with backtracking and print the final board. The program prints the board at every placement and backtrack step.

6.2 Theory

- **Backtracking:** build the solution one piece at a time (one queen per row). If a piece cannot be placed anywhere, undo the previous piece and try its next option. It is depth-first search on the state-space tree with a feasibility check that prunes dead branches early.
- **Attack rules:** two queens at (r1, c1) and (r2, c2) attack if r1 = r2, or c1 = c2, or |r1−r2| = |c1−c2| (same diagonal).
- **One queen per row** is forced (n queens, n rows, no two in a row), so we only choose a column for each row. That reduces the search space from choosing n squares out of n^2 to at most n^n, and pruning reduces it much further.
- Known facts: no solution for n = 2 or 3; n = 4 has 2 solutions; n = 8 has 92 solutions. With a *fixed* first queen a solution is not guaranteed (see dry run).

6.3 Approach

1. Create an n × n board of zeros (1 = queen).
2. Read the first queen's row and column (1-based), convert to 0-based, validate, and place it.
3. Make the list of remaining rows (every row except the first queen's row).
4. For each remaining row, try columns left to right; if is_safe, place a queen and recurse to the next remaining row.
5. If the recursion fails, remove the queen (backtrack) and try the next column. If no column works, return False to the previous row.
6. If all remaining rows are filled, print the final matrix; otherwise report that no solution exists for that starting position.

6.4 Code walkthrough

```
def is_safe(board, row, col, n):
    for i in range(n):                                  # whole row and whole column
        if board[row][i] == 1 or board[i][col] == 1:
            return False
    for dr, dc in [(-1,-1), (-1,1), (1,-1), (1,1)]:     # all 4 diagonal directions
        r, c = row + dr, col + dc
        while 0 <= r < n and 0 <= c < n:
            if board[r][c] == 1: return False
            r += dr; c += dc
    return True

def solve_nqueens(board, row_idx, remaining_rows, n):
    if row_idx == len(remaining_rows): return True      # all queens placed
    current_row = remaining_rows[row_idx]
    for col in range(n):
        if is_safe(board, current_row, col, n):
            board[current_row][col] = 1                 # place
            if solve_nqueens(board, row_idx + 1, remaining_rows, n): return True
            board[current_row][col] = 0                 # backtrack
    return False
```

- **Why check all four diagonal directions and the full column?** The fixed queen can be in *any* row, including below the row currently being filled. A normal solver that fills rows top to bottom only needs

to look upward; here queens may exist above and below, so the check must be full.

- **remaining_rows** is the list of rows still to fill, so the solver skips the fixed queen's row. `row_idx` indexes this list, not the board row.
- **global step** numbers each "[TRY / PLACE]" and "[BACKTRACK]" print so the whole search can be followed in the output.
- **Upfront checks:** n ≤ 0 is rejected; n = 2 and 3 are reported as having no solution immediately; row/column outside 1..n is rejected. n = 1 works (board with a single queen).
- Returns True as soon as *one* solution is found (it does not enumerate all solutions).

6.5 Dry run (verified): n = 4, first queen at row 1, column 2

1. Remaining rows: 2, 3, 4.
2. Row 2: col 1 is diagonal to (1,2); col 2 same column; col 3 diagonal; **col 4 safe** → place (2,4).
3. Row 3: **col 1 safe** (checked against (1,2) and (2,4)) → place (3,1).
4. Row 4: col 1 and col 2 are taken columns; **col 3 safe** → place (4,3). All rows done.

```
Final board (verified output)      Queens: (1,2) (2,4) (3,1) (4,3)
0  1  0  0
0  0  0  1
1  0  0  0
0  0  1  0
```

Same n = 4 but first queen at (1,1): every attempt eventually fails, the program backtracks through all options and prints *No solution exists with the first Queen placed at (1, 1)*. n = 6 with first queen at (2,1) also gives a valid board.

6.6 Complexity

Measure	Value	Reason
Time (search)	O(n!) worst case	Roughly n column choices for the first row and fewer for each later row (attacked columns and diagonals are excluded), giving factorial growth
Per safety check	O(n)	Scans row, column and diagonals
Overall (conservative)	O(n · n!)	Number of nodes × cost of is_safe
Space	$O(n^2)$ board + O(n) recursion depth	Board matrix plus at most n−1 stacked calls

6.7 Viva Q&A

Q1. What is backtracking?

A depth-first way of building a solution incrementally and undoing the last choice (backtracking) when it leads to a dead end.

Q2. How does backtracking improve on brute force?

Brute force tries all placements, about n^n for one-per-row, and checks at the end. Backtracking checks constraints at each step and abandons a partial board as soon as it is invalid.

Q3. How do you test the diagonal?

Two squares are on the same diagonal if |r1−r2| = |c1−c2|. This program walks outward in the four diagonal directions until the board edge.

Q4. What is the time complexity?

O(n!) in the worst case; each safety check is O(n), so about O(n · n!) overall.

Q5. Why is there no solution for n = 2 and n = 3?

On a 2×2 or 3×3 board every placement leaves two queens attacking each other (exhaustive check). The program prints this immediately instead of searching.

Q6. Why might a solution not exist for a fixed first queen?

The fixed queen removes a row, a column and two diagonals from consideration, which may make the rest impossible. Example: n = 4 with queen at (1,1).

Q7. How many solutions are there for n = 8?

92 total (12 fundamentally distinct, up to rotation and reflection).

Q8. How would you make is_safe faster?

Keep boolean arrays for used columns, and for the two diagonal families (r+c and r−c). The check becomes O(1). Bitmasks make it even faster.

Q9. Why is the full column/row check needed here but not in the standard algorithm?

In the standard algorithm queens exist only in earlier rows, so only upward checks are needed. With a pre-placed queen that may be below, all directions must be checked.

Q10. Why is one queen per row guaranteed?

n queens in n rows with no two sharing a row forces exactly one per row, which is why the board needs only a column choice per row.

Q11. Why return True immediately? How would you count all solutions?

The task needs just one solution. To count all, do not return on success; increment a counter, backtrack and continue.

Q12. Backtracking vs Branch and Bound vs brute force?

Brute force: no pruning. Backtracking: prune infeasible partial solutions (constraints). B&B: prune using a bound on the objective value (optimisation).

Q13. What is the state-space tree here?

Root = board with the fixed queen; each level assigns a column to the next remaining row; leaves are complete boards or dead ends.

Q14. What is the purpose of printing each step?

To show the search itself: placements, failures and backtracking, which is the essence of the method.

Q15. Is N-Queens hard in general?

Finding one solution for plain N-Queens is easy (constructive formulas exist). Completing an arbitrary partially filled board is known to be hard (NP-complete), which is why a fixed first queen can make search expensive.

Part 7: Where an examiner may probe the code (honest answers)

Observation	How to answer
Recursive Fibonacci is called inside a loop for every i (Practical 1).	Yes, that is how the whole series is printed. Total work is the sum of calls for fib(0)..fib(n−1), which is still exponential (dominated by the last terms). A memoised version would fix this.
Time shown is measured in microseconds and includes printing.	Timing includes I/O, so absolute numbers are noisy. The comparison still shows the large gap between $O(2^n)$ and O(n). For a clean benchmark, time without printing and repeat runs.
Huffman tree and codes exist only while the program runs (no file output, no header stored).	This practical demonstrates the algorithm. A real compressor would also store the tree or frequency table and pack bits into bytes.
Huffman encoded output is a string of '0' and '1' characters, not real bits.	Correct, for readability. In a real file you would pack 8 bits per byte, and the compression gain would be real.
Fractional knapsack is O(n log n) because of sorting.	True; O(n) is possible with weighted median but unnecessary here.
DP keeps the whole table.	Needed to trace back selected items. If only the maximum value is needed, one array of size W+1 suffices.
Knapsack programs accept only integers.	DP needs integer capacity as table index. B&B could handle decimals, but input is kept integer for consistency with the DP version so results can be compared.
B&B here is depth-first, not the textbook best-first.	Intentional for simplicity and O(n) memory. Best-first would use a priority queue ordered by bound; same bound, different exploration order.
N-Queens complexity is quoted as O(n!) although is_safe is O(n).	O(n!) counts search nodes; including the O(n) safety check gives O(n·n!). Both statements are consistent.
N-Queens is_safe also checks the row, which is redundant for remaining rows.	True: remaining rows are empty when filled, so only the column and diagonals matter. The extra check is harmless and keeps is_safe general.
Only one N-Queens solution is produced.	By design the first found solution is returned. To list all, continue after success and count.

Part 8: One-page revision sheet

Practical	Key idea (say this first)	Formula / rule	Test input and answer
1 Fibonacci and Hanoi	Recursion recomputes subproblems; iteration reuses.	T(n)=T(n−1)+T(n−2)+1; Hanoi 2^n-1	7 terms: 0 1 1 2 3 5 8; 3 disks: 7 moves
2 Huffman	Merge two least-frequent nodes using a min-heap; prefix codes.	Cost = Σ freq × length; O(k log k)	AAABBC → A=0, C=10, B=11; 000111110
3 Fractional knapsack	Sort by value/weight, fill greedily, split last item.	O(n log n)	60/10, 100/20, 120/30, W=50 → 240.00
4 0/1 Knapsack DP	Table of best value for (items, capacity); take or skip.	dp[i][w]=max(dp[i−1][w], v+dp[i−1][w−wt]); O(nW)	Same data → 220, items 2 and 3
5 0/1 Knapsack B&B	DFS include/exclude; prune if fractional upper bound ≤ best.	bound = profit + greedy fill + fraction	Same data → 220 (best path 160 → 180 → 220)
6 N-Queens	One queen per row, check column/diagonals, undo on failure.	|r1−r2| = |c1−c2| means attack; O(n!)	n=4, (1,2) → solved; n=4, (1,1) → none

Last-minute tip: for any practical, answer in this order: (1) what problem, (2) which paradigm and why, (3) the core step of the algorithm, (4) the complexity with a reason, (5) one edge case your code handles. Then run the sample input and explain the output.